COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	9/14/2026	**DATE SUBMITTED:**	9/16/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 **Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of shell, merge and quick sort.

Preparation Task:

- Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 8-1 in section 6.
- Create a header file for the implementation of the different sorting algorithms.
- Import this header file into your main.cpp

Code + Console Screenshot

main.cpp

```
6   const int SIZE = 100;
7
8   int main(){
9
10      // Array of size 100 to hold random integers
11      int arr[SIZE];
12      srand(time(0));
13
14      for (int i = 0; i < SIZE; i++){
15          arr[i] = rand()%100;
16      }
17
18      std::cout << "Original Array:" << std::endl;
19      displayArray(arr, SIZE);
```

Header

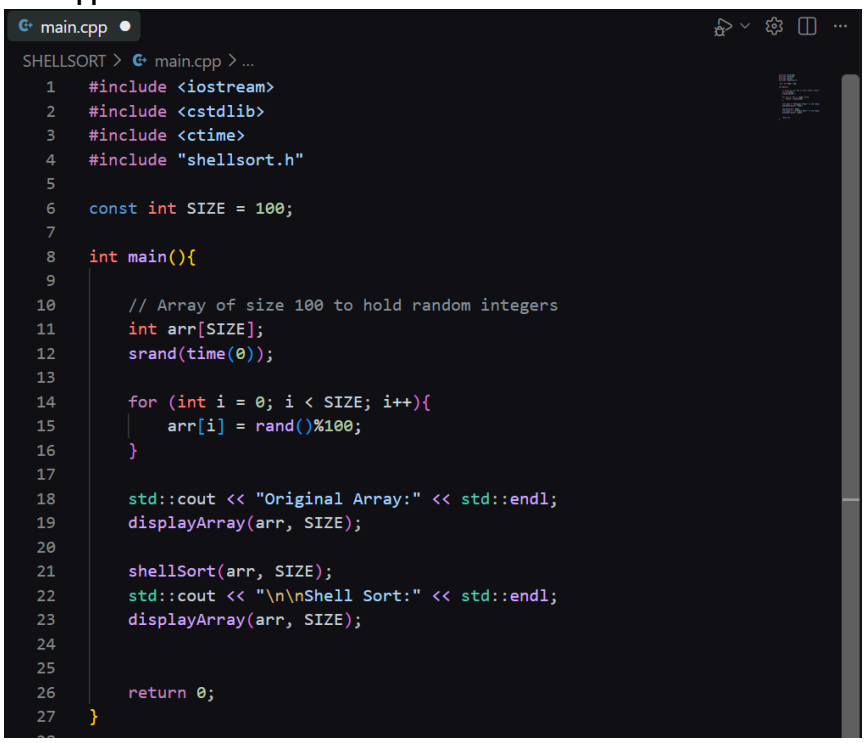
```
5   void displayArray(int arr[], int size)
6   {
7       for (int i = 0; i < size; i++){
8           std::cout << arr[i] << " ";
9       }
10  }
```

Console Screenshot:

```
Original Array:
8 48 49 21 94 61 93 97 75 95 38 32 11 36 6 57 60 0 81 60 31 38 70 18 43 20 48 34 88 84 6 48 16 31 86 44 76 9
5 15 86 93 77 45 9 21 25 33 74 7 76 57 19 75 5 13 19 32 39 91 27 48 71 8 74 50 70 55 89 87 97 69 2 59 71 6 7
7 1 79 31 79 64 31 52 15 67 60 24 19 11 27 15 76 43 7 92 92 72 40 19 15
```

Observations	The code creates an array with 100 elements and fills it with random integer values from 0 to 99. The srand(time(0)) is used so that the values are randomized each time the program runs. The generated array is not sorted yet but will be used to test the Shell Sort, Merge Sort and Quick Sort.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	**main.cpp**  **shellsort.h**

shellsort.h

SHELLSORT > shellsort.h > ...

```
#ifndef SHELLSORT_H
#define SHELLSORT_H
#include <iostream>

void displayArray(int arr[], int size)
{
    for (int i = 0; i < size; i++){
        std::cout << arr[i] << " ";
    }
}

/*

A.1. Shell Sort
General Algorithm
The general algorithm for shell sort is given below.
shellSort(array, size)
for interval i <- size/2n down to 1
for each interval "i" in array
sort all the elements at interval "i"
end shellSort

*/

// Shell Sort (CREDITS TO PROGRAMIZ)

void shellSort(int array[], int n)
{
    // Rearrange elements at each n/2, n/4, n/8, ... intervals
    for (int interval = n / 2; interval > 0; interval /= 2) {
        for (int i = interval; i < n; i += 1) {
            int temp = array[i];
            int j;

            for (j = i; j >= interval && array[j - interval] > temp; j -= int
                array[j] = array[j - interval];
            }

            array[j] = temp;
        }
    }
}

#endif
```

Console Screenshot:

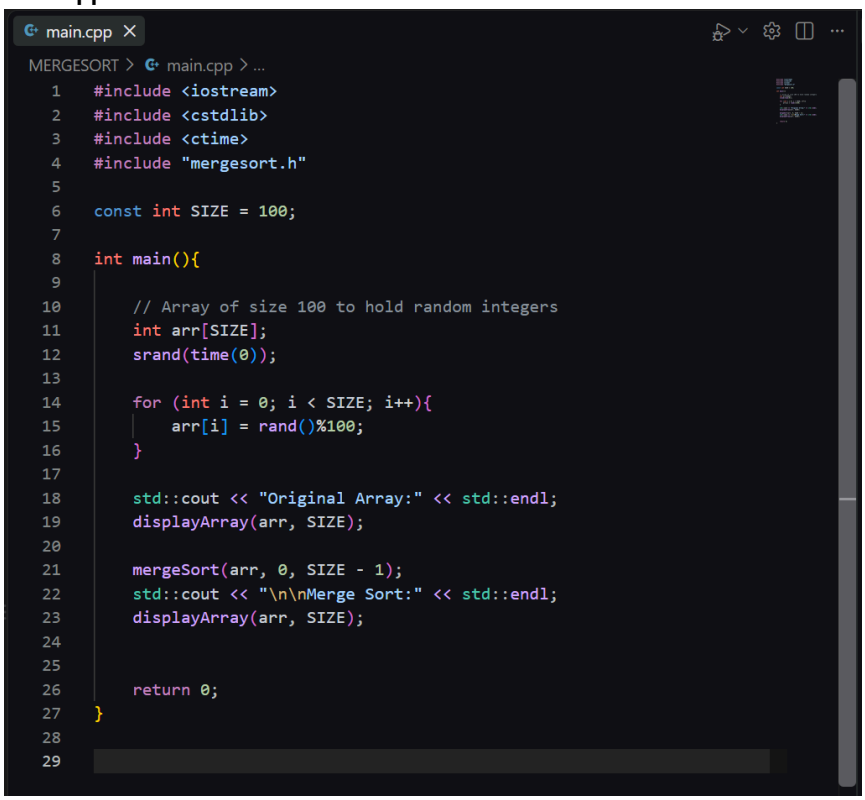
```
Original Array:
82 29 89 63 85 3 81 64 0 97 12 56 37 89 63 57 88 18 64 60 80 26 80 61 26 10 8 46 73 31 77 21 74 73 78 97 90
96 83 92 55 72 76 34 64 81 80 41 35 85 90 26 10 70 31 35 30 75 94 97 66 69 52 4 31 21 92 40 42 76 22 58 47 2
7 66 8 80 36 84 41 57 69 85 53 99 20 76 71 10 98 14 16 12 48 63 51 63 14 38 81

Shell Sort:
0 3 4 8 8 10 10 10 12 12 14 14 16 18 20 21 21 22 26 26 26 27 29 30 31 31 31 34 35 35 36 37 38 40 41 41 42 46
 47 48 51 52 53 55 56 57 57 58 60 61 63 63 63 63 64 64 64 66 66 69 69 70 71 72 73 73 74 75 76 76 76 77 78 80
 80 80 80 81 81 81 82 83 84 85 85 85 88 89 89 90 90 92 92 94 96 97 97 97 98 99
```

Observations	The Shell Sort algorithm performs the sorting operation on the array using different gaps instead of comparing elements adjacent to each other directly. The interval used in the Shell Sort code begins with half the size of the array and continues getting divided by 2 until it becomes equal to 1. This helps in moving elements, which are distant from their right position, nearer to the desired location. The decreasing size of gaps in the array makes the array organized;

	once the interval becomes 1, the sorting process works as an insertion sort algorithm to arrange all elements in ascending order.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	**main.cpp**  **mergesort.h**

mergesort.h

MERGESORT > mergesort.h > ...

```
#ifndef MERGESORT_H
#define MERGESORT_H
#include <iostream>

void displayArray(int arr[], int size)
{
    for (int i = 0; i < size; i++){
        std::cout << arr[i] << " ";
    }
}

/*

A.2. Merge Sort
General Algorithm
The general pseudo-code for the merge sort technique is given below.
Declare an array Arr of length N
If N=1, Arr is already sorted
If N>1,
Left = 0, right = N-1
Find middle = (left + right)/2
Call merge_sort(Arr,left,middle) =>sort first half recursively
Call merge_sort(Arr,middle+1,right) => sort second half recursively
Call merge(Arr, left, middle, right) to merge sorted arrays in above steps.
Exit

*/

// Merge Sort (CREDITS TO PROGRAMIZ)

// Merge two subarrays L and M into arr
void merge(int arr[], int p, int q, int r)
{
    // Create L ← A[p..q] and M ← A[q+1..r]
    int n1 = q - p + 1;
    int n2 = r - q;

    int* L = new int[n1];
    int* M = new int[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[p + i];

    for (int j = 0; j < n2; j++)
        M[j] = arr[q + 1 + j];

    // Maintain current index of sub-arrays and main array
```

```
    int i = 0, j = 0, k = p;

    // Until we reach either end of either L or M, pick larger among
    // elements L and M and place them in the correct position at A[p.
    while (i < n1 && j < n2){
        if (L[i] <= M[j]){
            arr[k] = L[i];
            i++;
        }
        else{
            arr[k] = M[j];
            j++;
        }
        k++;
    }

    // When we run out of elements in either L or M,
    // pick up the remaining elements and put in A[p..r]
    while (i < n1){
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2){
        arr[k] = M[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] M;
}

// Divide the array into two subarrays, sort them and merge them
void mergeSort(int arr[], int l, int r)
{
    if (l < r){
        // m is the point where the array is divided into two subarray
        int m = l + (r - l) / 2;

        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        // Merge the sorted subarrays
        merge(arr, l, m, r);
    }
}

#endif
```

Console Screenshot:

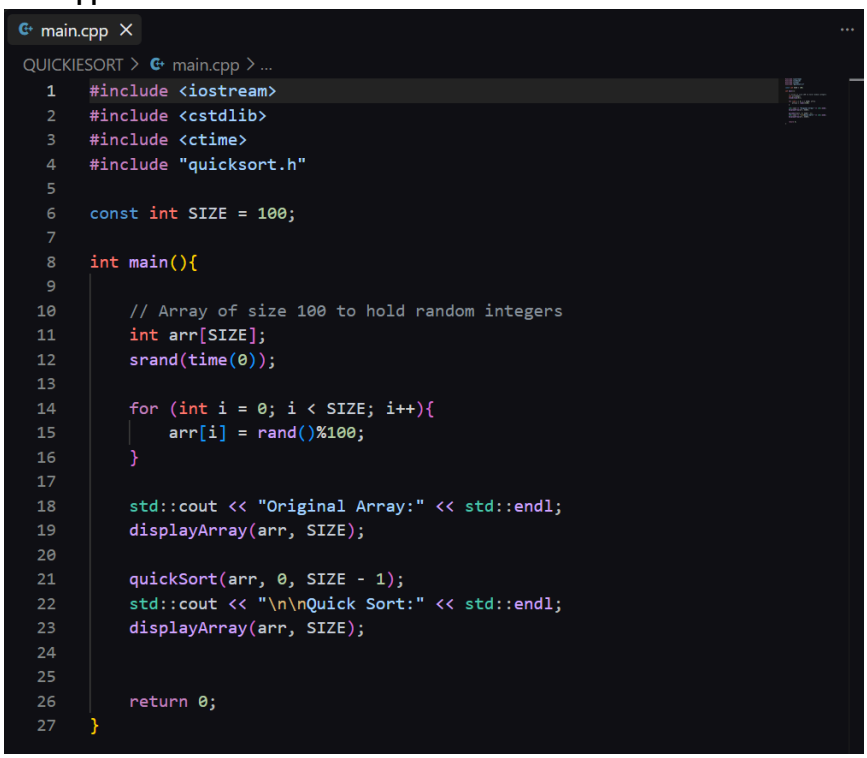
```
Original Array:
89 78 60 26 81 65 25 23 31 20 48 35 83 32 72 58 96 97 99 18 62 62 58 62 80 85 39 8 35 38 14 16 99 99 64 48 3
9 51 20 45 19 11 66 91 65 68 12 67 69 26 50 7 14 41 14 0 20 2 53 88 63 0 33 79 83 43 12 67 82 20 83 41 96 43
 38 49 21 70 3 41 0 7 93 70 86 11 15 64 20 20 41 0 81 0 90 90 3 58 51 67

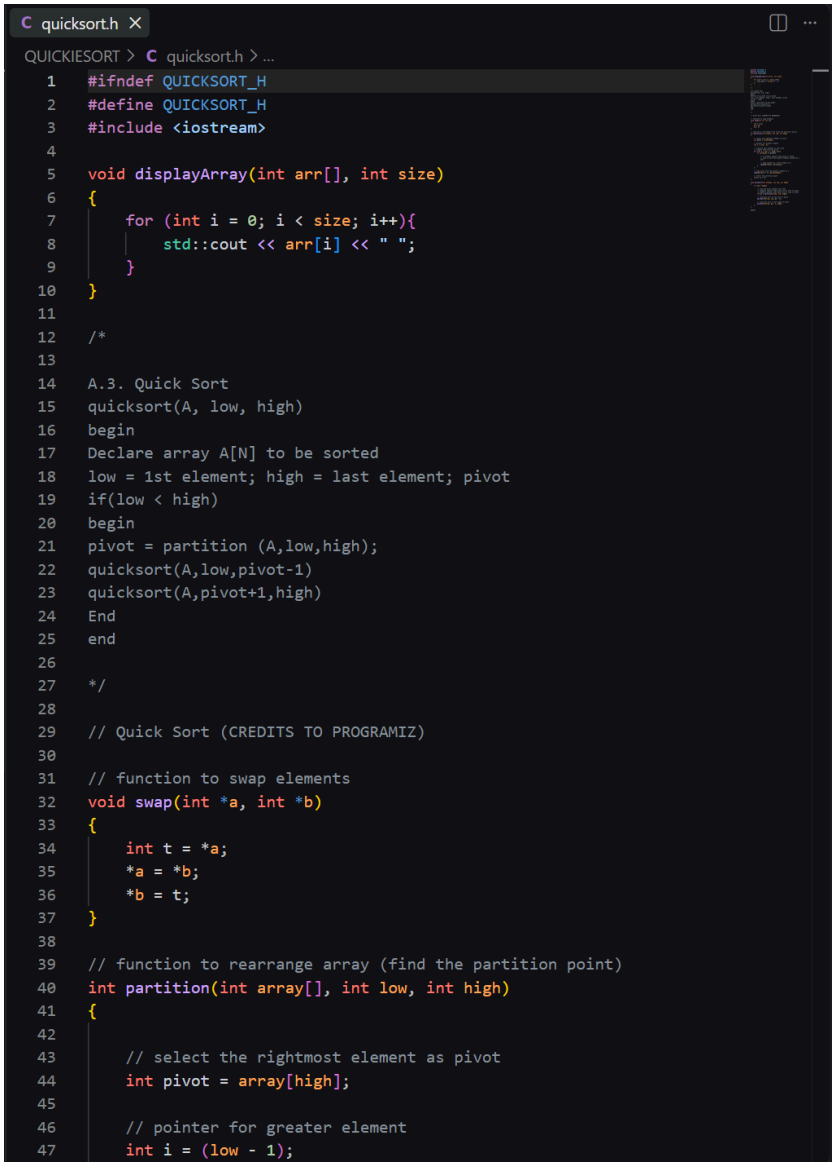
Merge Sort:
0 0 0 0 0 2 3 3 7 7 8 11 11 12 12 14 14 14 15 16 18 19 20 20 20 20 20 21 23 25 26 26 31 32 33 35 35 38 38
 39 39 41 41 41 41 43 43 45 48 48 49 50 51 51 53 58 58 58 60 62 62 62 63 64 64 65 65 66 67 67 67 68 69 70 70
 72 78 79 80 81 81 82 83 83 83 85 86 88 89 90 90 91 93 96 96 97 99 99 99
```

Observations	For the merge sort algorithm, the 100 element array is divided to create subarrays and will continue to keep making subarrays until it makes arrays that contain only one element. After which, the smaller subarrays are compared and

	merged back into larger and ordered arrays. The function mergeSort() makes the division and recursive calls for sorting, and the merge() function does the merging of the smaller arrays into larger arrays. This process continues until all the elements are merged into one sorted array in ascending order.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	**main.cpp**  **quicksort.h**


quicksort.h
QUICKIESORT > quicksort.h > ...
#ifndef QUICKSORT_H
#define QUICKSORT_H
#include <iostream>
void displayArray(int arr[], int size)
{
for (int i = 0; i < size; i++){
std::cout << arr[i] << " ";
}
}
/*
A.3. Quick Sort
quicksort(A, low, high)
begin
Declare array A[N] to be sorted
low = 1st element; high = last element; pivot
if(low < high)
begin
pivot = partition (A,low,high);
quicksort(A,low,pivot-1)
quicksort(A,pivot+1,high)
End
end
*/
// Quick Sort (CREDITS TO PROGRAMIZ)
// function to swap elements
void swap(int *a, int *b)
{
int t = *a;
*a = *b;
*b = t;
}
// function to rearrange array (find the partition point)
int partition(int array[], int low, int high)
{
// select the rightmost element as pivot
int pivot = array[high];
// pointer for greater element
int i = (low - 1);

```
48
49      // traverse each element of the array
50      // compare them with the pivot
51      for (int j = low; j < high; j++){
52          if (array[j] <= pivot){
53
54              // if element smaller than pivot is found
55              // swap it with the greater element pointed by i
56              i++;
57
58              // swap element at i with element at j
59              swap(&array[i], &array[j]);
60          }
61      }
62
63      // swap pivot with the greater element at i
64      swap(&array[i + 1], &array[high]);
65
66      // return the partition point
67      return (i + 1);
68  }
69
70  void quickSort(int array[], int low, int high)
71  {
72      if (low < high){
73
74          // find the pivot element such that
75          // elements smaller than pivot are on left of pivot
76          // elements greater than pivot are on righ of pivot
77          int pi = partition(array, low, high);
78
79          // recursive call on the left of pivot
80          quickSort(array, low, pi - 1);
81
82          // recursive call on the right of pivot
83          quickSort(array, pi + 1, high);
84      }
85  }
86
87  #endif
```

Console Screenshot:

```
Original Array:
39 91 86 97 97 52 10 51 38 70 82 0 7 83 32 98 93 0 64 3 87 77 21 43 88 27 0 69 86 34 10 25 94 93 6 34 49 67
8 52 22 13 37 53 73 90 34 15 3 50 39 30 59 41 5 20 89 50 30 20 89 71 32 47 30 76 80 24 7 48 92 15 65 38 44 8
 82 80 18 97 26 16 35 61 56 8 60 74 83 11 24 70 26 16 8 81 81 31 25 70

Quick Sort:
0 0 0 3 3 5 6 7 7 8 8 8 8 10 10 11 13 15 15 16 16 18 20 20 21 22 24 24 25 25 26 26 27 30 30 30 31 32 32 34 3
4 34 35 37 38 38 39 39 41 43 44 47 48 49 50 50 51 52 52 53 56 59 60 61 64 65 67 69 70 70 70 71 73 74 76 77 8
0 80 81 81 82 82 83 83 86 86 87 88 89 89 90 91 92 93 93 94 97 97 97 98
```

Observations	In Quick Sort, sorting is carried out using one particular element, which is called the pivot element, and it is always the rightmost element. To do that, the program uses the index j to examine every element of the array in relation to the pivot element. There are three conditions in our code: C1 – is when the current value of the element at j index is bigger than the pivot, so j index just proceeds to the next element; C2 – when the current value at j index is smaller than the pivot element, so i index is increased and this element is placed in the right place to the left of the pivot; C3 – when j==pivot index which means it reaches the pivot element, i index is increased, and the pivot element is placed to the right place.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms?

Yes, we can sort the left and right sub list from the partition method in quick sort using other sorting algorithms. This can be sorted by using the insertion sort algorithm when we want to sort the smaller sublists instead of using quick sort for the entire code.

Demonstrate an example:

CODE:

main.cpp

main.cpp

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "problem1.h"

const int SIZE = 100;

int main(){

    // Array of size 100 to hold random integers
    int arr[SIZE];
    srand(time(0));

    for (int i = 0; i < SIZE; i++){
        arr[i] = rand()%100;
    }

    std::cout << "Original Array:" << std::endl;
    displayArray(arr, SIZE);

    quickSort(arr, 0, SIZE - 1);

    std::cout << "\n\nQuick Sort with Insertion Sort:" << std::endl;
    displayArray(arr, SIZE);


    return 0;
}
```

problem1.h

problem1.h

```
#ifndef PROBLEM1_H
#define PROBLEM1_H
#include <iostream>

void displayArray(int arr[], int size)
{
    for (int i = 0; i < size; i++){
        std::cout << arr[i] << " ";
    }
}

// Insertion Sort
void insertionSort(int arr[], int low, int high)
{
    for (int i = low + 1; i <= high; i++){
        int temp = arr[i];
        int j = i - 1;

        while (j >= low && arr[j] > temp){
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = temp;
    }
}

// Partition
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++){
        if (arr[j] <= pivot){
            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

```

```
48      return i + 1;
49  }
50
51  // Quick Sort using Insertion Sort for sublists
52  void quickSort(int arr[], int low, int high)
53  {
54      if (low < high){
55          int pivot = partition(arr, low, high);
56
57          // Use Insertion Sort for the left sublist
58          insertionSort(arr, low, pivot - 1);
59
60          // Use Insertion Sort for the right sublist
61          insertionSort(arr, pivot + 1, high);
62      }
63  }
64
65  #endif
```

OUTPUT:

```
Original Array:
83 34 25 81 4 24 25 46 34 19 76 79 65 48 0 41 77 56 91 16 72 67 73 30 88 68 84 1 27 3 68 86 67 23 62 18 95 1
7 59 84 15 25 21 70 23 32 6 84 63 66 48 83 9 8 36 56 48 76 85 10 72 0 71 94 90 32 24 25 56 21 39 29 0 70 24
88 58 56 56 72 84 9 69 59 81 60 92 77 96 28 4 60 98 85 50 14 18 55 91 78

Quick Sort with Insertion Sort:
0 0 0 1 3 4 4 6 8 9 9 10 14 15 16 17 18 18 19 21 21 23 23 24 24 24 25 25 25 25 27 28 29 30 32 32 34 34 36 39
 41 46 48 48 48 50 55 56 56 56 56 56 58 59 59 60 60 62 63 65 66 67 67 68 68 69 70 70 71 72 72 72 73 76 76 77
 77 78 79 81 81 83 83 84 84 84 84 85 85 86 88 88 90 91 91 92 94 95 96 98
```

Analysis

The program shows that the left and right sublists generated by the Quick Sort algorithm's partition function can be sorted by means of another sorting algorithm The partition() function takes the rightmost number as a pivot and rearranges the array such that all elements less than the pivot are placed to the left and those greater to the right. The quickSort() function finds the pivot position and invokes the insertionSort() function instead of making a recursive call to the quickSort() function for the two sublists. The insertionSort() function sorts the elements in the given range of low and high indexes by comparing and moving them to the appropriate place.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

CODE:

main.cpp

main.cpp

problem2 > main.cpp > ...

```cpp
#include <iostream>
#include "problem2.h"

const int SIZE = 15;

int main(){

    int arr[SIZE] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 7

    std::cout << "Original Array:" << std::endl;
    displayArray(arr, SIZE);

    quickSort(arr, 0, SIZE - 1);

    std::cout << "\n\nQuick Sort:" << std::endl;
    displayArray(arr, SIZE);

    return 0;
}

```

problem2.h

problem2.h

problem2 > problem2.h > ...

```cpp
#ifndef PROBLEM2_H
#define PROBLEM2_H
#include <iostream>

void displayArray(int arr[], int size)
{
    for (int i = 0; i < size; i++){
        std::cout << arr[i] << " ";
    }
}

/*

A.3. Quick Sort
quicksort(A, low, high)
begin
Declare array A[N] to be sorted
low = 1st element; high = last element; pivot
if(low < high)
begin
pivot = partition (A,low,high);
quicksort(A,low,pivot-1)
quicksort(A,pivot+1,high)
End
end

*/

// Quick Sort (CREDITS TO PROGRAMIZ)

// function to swap elements
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

// function to rearrange array (find the partition point)
int partition(int array[], int low, int high)
{

    // select the rightmost element as pivot
    int pivot = array[high];

    // pointer for greater element
    int i = (low - 1);
```

```
48
49      // traverse each element of the array
50      // compare them with the pivot
51      for (int j = low; j < high; j++){
52          if (array[j] <= pivot){
53
54              // if element smaller than pivot is found
55              // swap it with the greater element pointed by i
56              i++;
57
58              // swap element at i with element at j
59              swap(&array[i], &array[j]);
60          }
61      }
62
63      // swap pivot with the greater element at i
64      swap(&array[i + 1], &array[high]);
65
66      // return the partition point
67      return (i + 1);
68  }
69
70  void quickSort(int array[], int low, int high)
71  {
72      if (low < high){
73
74          // find the pivot element such that
75          // elements smaller than pivot are on left of pivot
76          // elements greater than pivot are on righ of pivot
77          int pi = partition(array, low, high);
78
79          // recursive call on the left of pivot
80          quickSort(array, low, pi - 1);
81
82          // recursive call on the right of pivot
83          quickSort(array, pi + 1, high);
84      }
85  }
86
87  #endif
```

OUTPUT:

QUESTIONS:

What sorting algorithm will give you the fastest time performance?

Quick Sort would be my choice as it is quite efficient in sorting an array such as the one having 15 numbers. Quick sort makes use of a pivot to partition the array in order to allow the elements to be sorted efficiently. Quick sort has O(N log N) average time complexity.

Why can merge sort and quick sort have O(N • log N) for their time complexity?
Merge Sort and Quick Sort can have O(N • log N) time complexity because they use the divide and conquer approach. The array is divided into smaller parts, creating about log N levels of division. At each level, the elements are processed, compared, or rearranged in about N operations. Therefore, N operations repeated over log N levels results in O(N • log N).

Conclusion:

From the activity, I learnt about Shell Sort, Merge Sort, and Quick Sort as algorithms that use different techniques to sort data. From Shell Sort, I learnt about gap-based sorting; Merge Sort uses dividing and merging of already sorted parts of the array; Quick Sort uses partitioning to sort data based on the pivot. Through the activity, I gained knowledge on how to apply these sorting algorithms through the provided pseudocode and complete the task.This code is based a lot on the Programiz samples, through which I came to know how Shell Sort, Merge Sort, and Quick Sort work in C++. I got an idea about sorting done by each algorithm and their respective pseudocodes.In supplementary activities, I got to learn that sorting algorithms can be combined; for instance, in Quick Sort, the sublists are sorted using Insertion Sort. I got to know why Merge Sort and Quick Sort algorithms are said to have time complexities of O(N log N). Overall, I believe that I performed pretty well in this activity since I could apply the algorithms and learn how they function. But there is still a lot for me to learn in regards to partitioning, recursion, and sorting time complexity of each algorithm and how I can determine what algorithm to sort specific data.

REFERENCES:
Programiz. (n.d.). ***Shell sort algorithm*****. Programiz.** https://www.programiz.com/dsa/shell-sort

Programiz. (n.d.). ***Merge sort algorithm*****. Programiz.** https://www.programiz.com/dsa/merge-sort

Programiz. (n.d.). ***Quick sort algorithm*****. Programiz.** https://www.programiz.com/dsa/quick-sort

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini, QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, formatting, summarization, code assistance, and explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to explain the code properly and everything else was referenced from programiz website to be able to complete the task that needed to be completed.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.